Data Manipulation

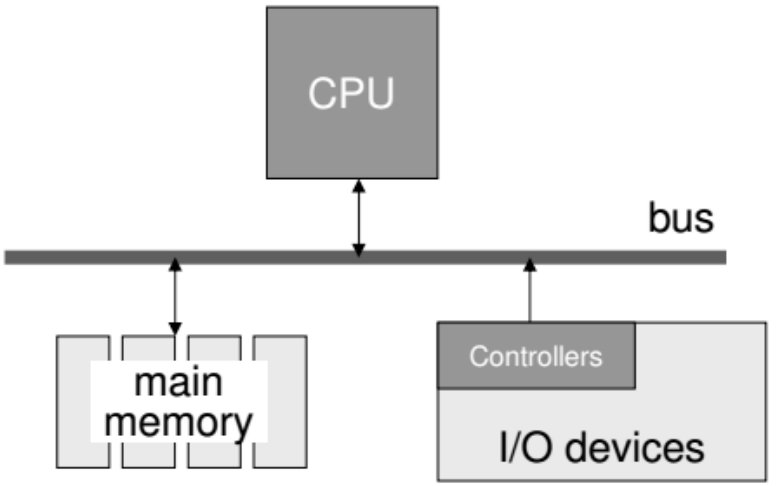
Lecture 4

Data Manipulation

- Computer Architecture
- Full Adder Circuit
- Machine Language
- Program Execution
- Arithmetic/Logic Instructions
- Communicating with Other Devices
- Program Data Manipulation
- Other Architectures

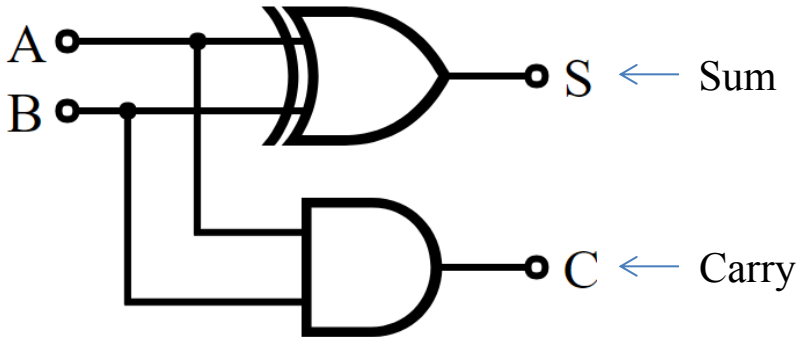
Computer Architecture

- Central Processing Unit (CPU) or processor
  - Arithmetic/Logic unit
  - Control unit
  - Registers
    - General purpose
    - Special purpose
  - Cache memory
- Main Memory
- I/O devices



Half Adder

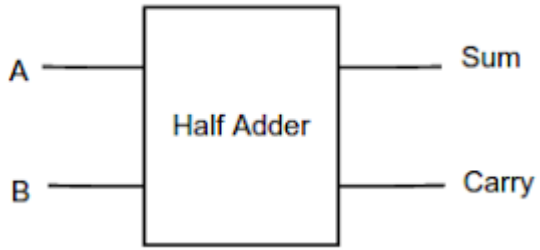
Inputs



Truth Table

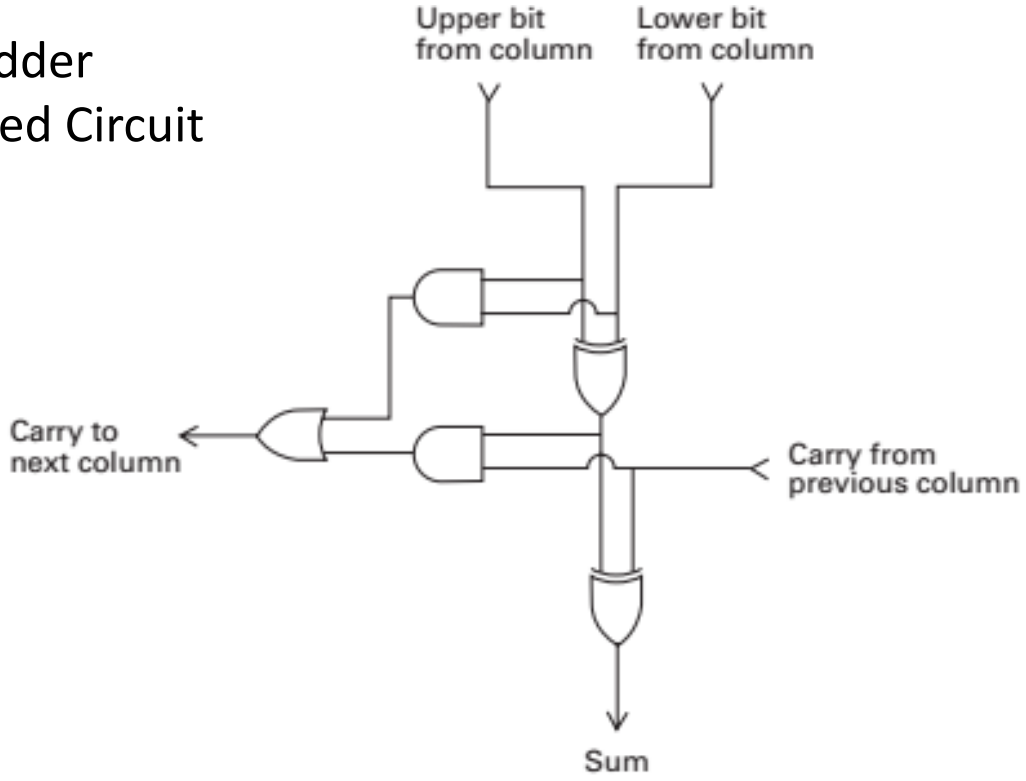
Inputs		Outputs	
A	*B*	*C*	*S*
0	0	0	0
1	0	0	1
0	1	0	1
1	1	1	0

Block Diagram

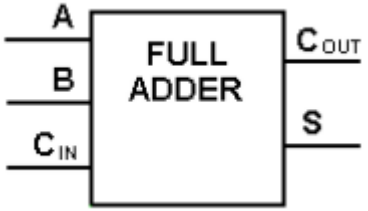


Single Column Adder (Full Adder)

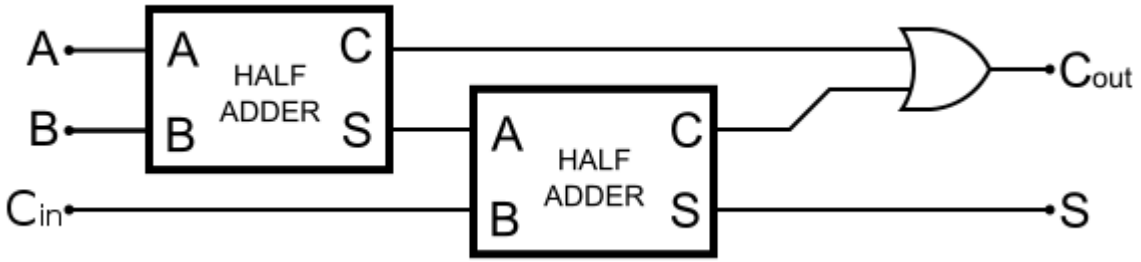
Full Adder
Detailed Circuit



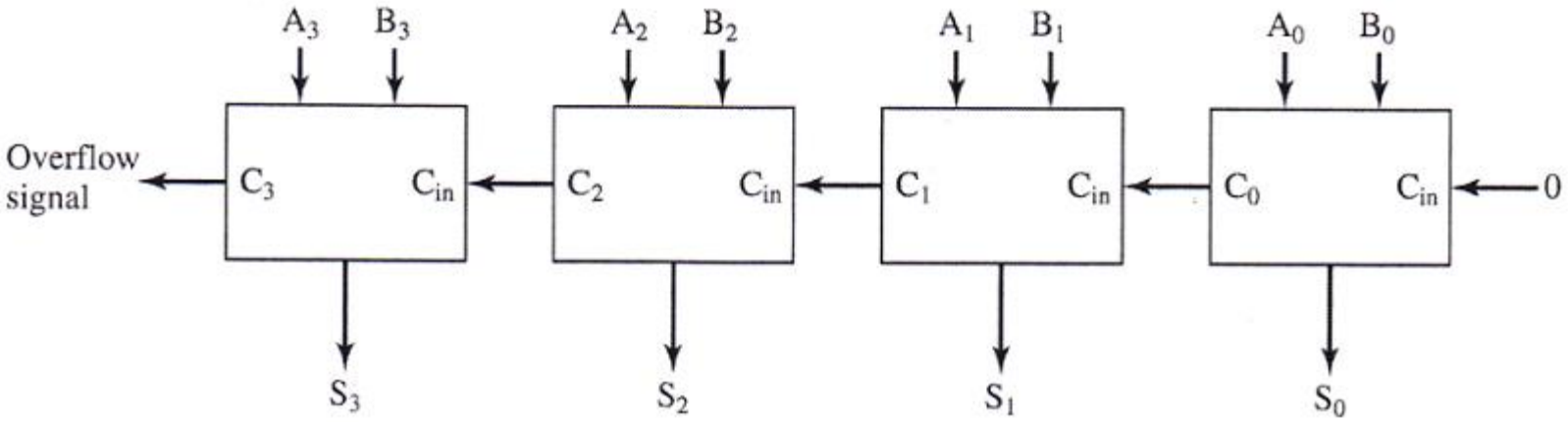
Full Adder Block
Diagram



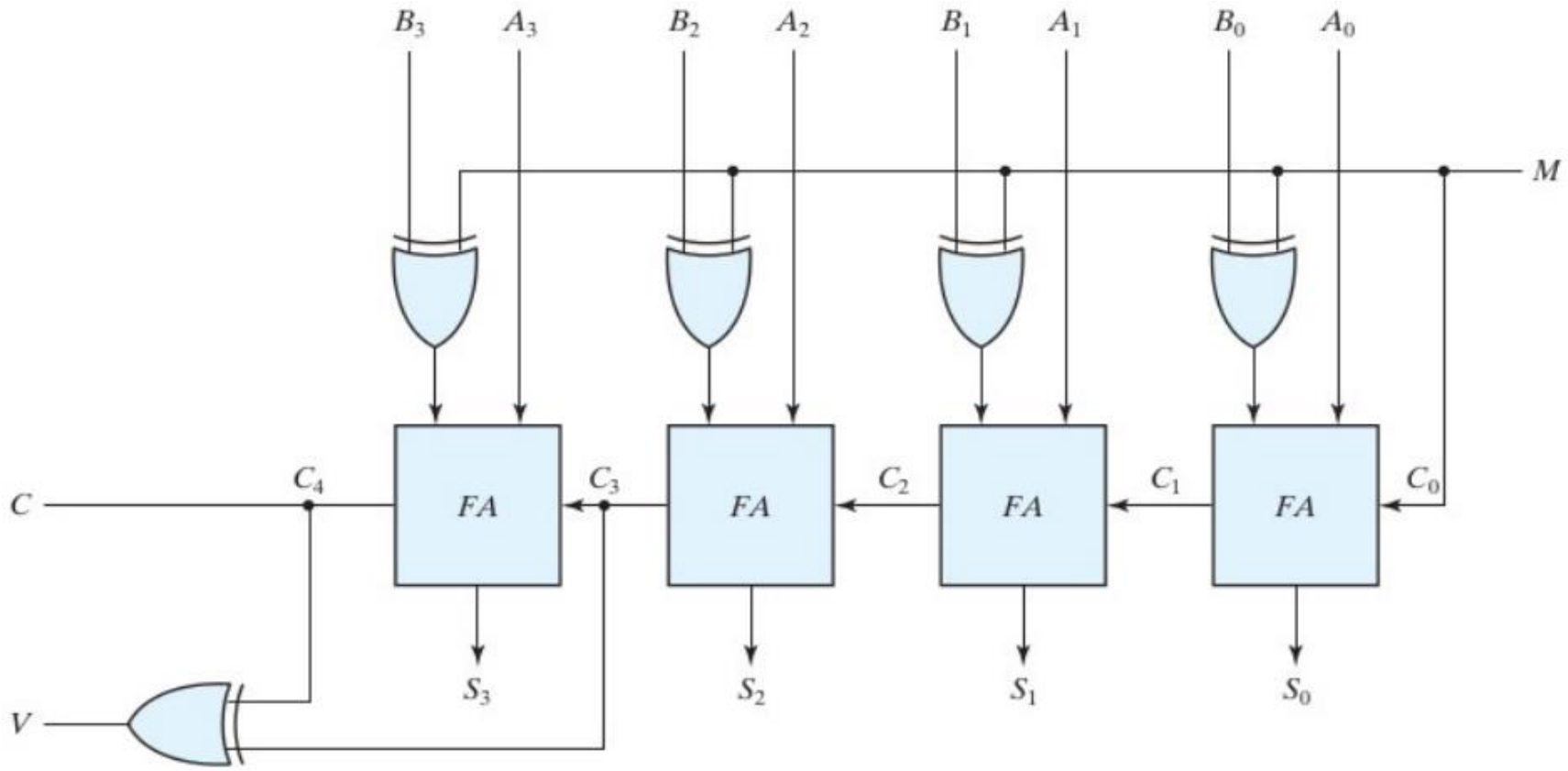
Full Adder With
Half-Adders



Multi-Column Adder

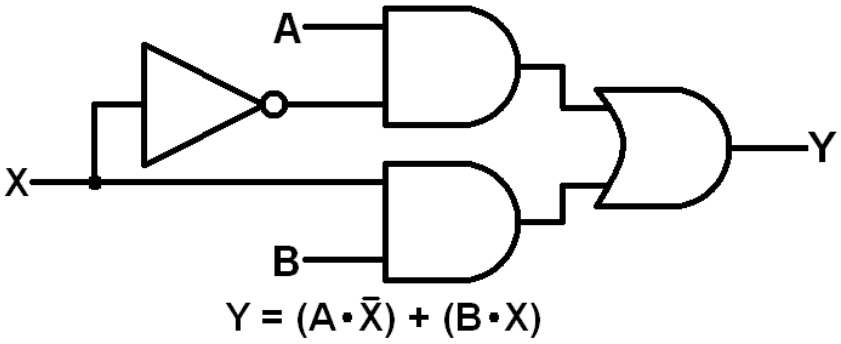


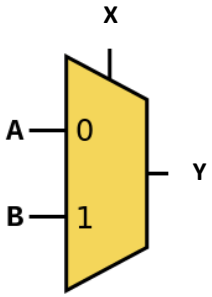
Multi-Column Adder/Subtractor



Multiplexor

- Multiplexor (MUX) is a device which allows selecting the input signal out from multiple source.
- X - select line (0 - input A is relayed, 1 - input B is relayed)
- A and B are input lines
- Y is an output line



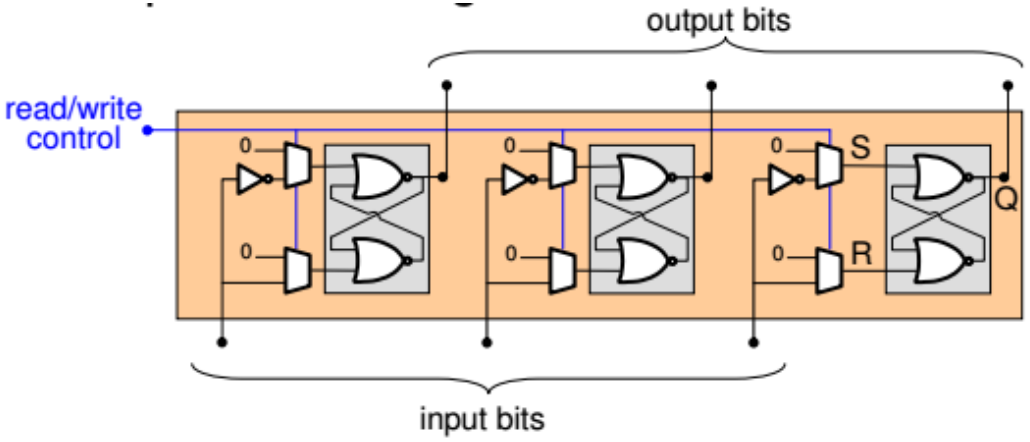


2-to-1 multiplexor

Select Input (X)	Input A	Input B	Output Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

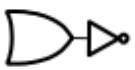
Register

- A register is a special memory cell inside the CPU that can store an n-bit data
  - For typical CPUs, n is 8, 16, 32 or 64
  - Registers are used to store intermediate computation results
- An n-bit register is composed of a group of n flip-flops
  - Example: a 3 bit register



The truth table of the flip-flop is

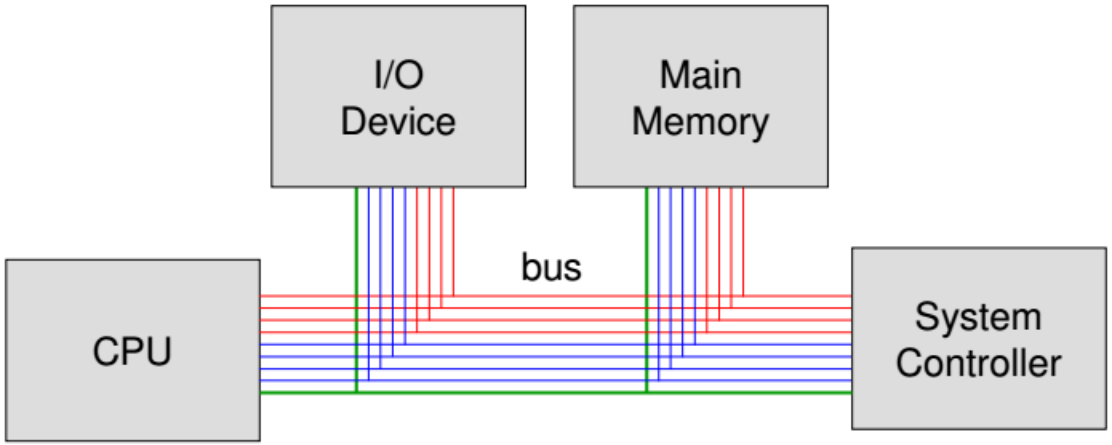
S	R	Q
0	0	Previous value
1	0	0
0	1	1
1	1	Don't care

Note:  is denoted as  , it is called a NOR gate.  is a 2-to-1 multiplexor.

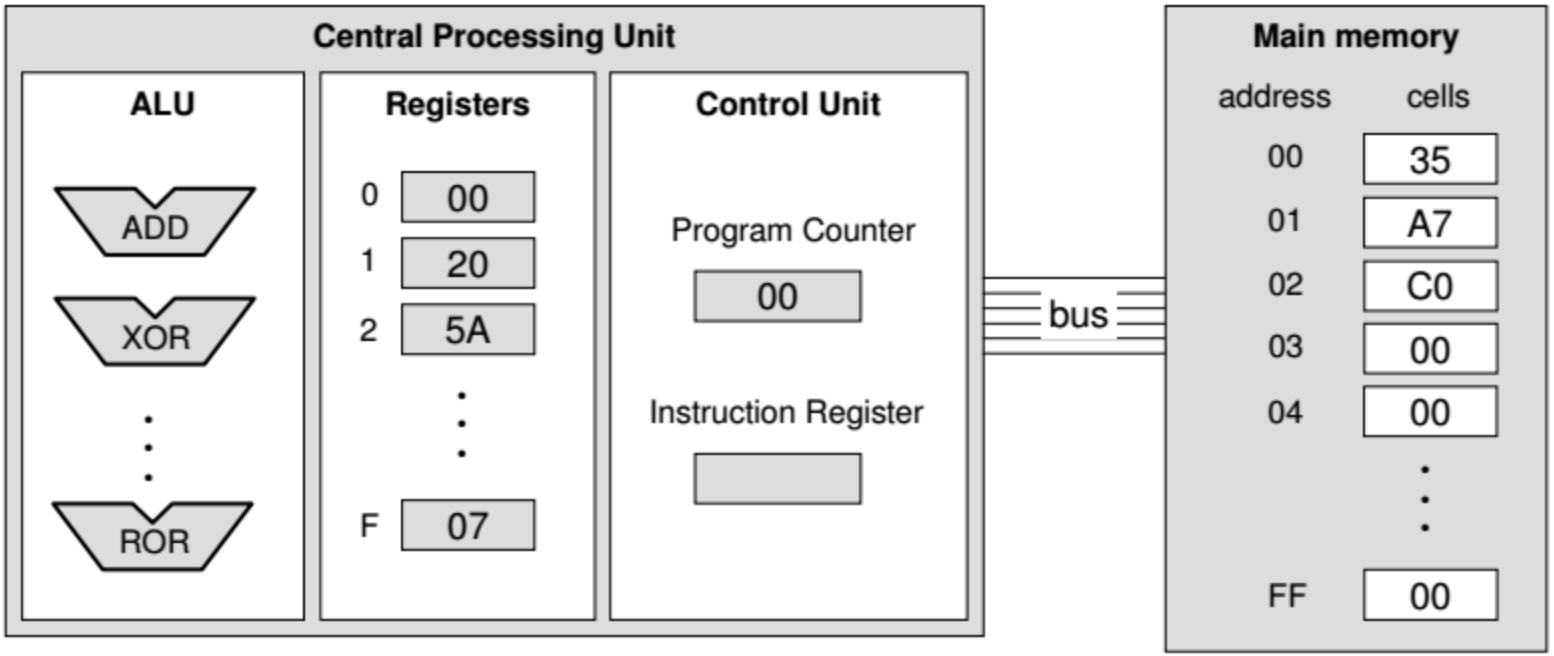
3/34

Bus

- A bus is a collection of wires to connect a computer's CPU, memory and I/O devices
- Bus has to carry 3 types of information:
  - Address of memory cell requested
  - Signal to read/write
  - Payload of data to write
- Information is carried over buss following ***handshaking rules***



Simple CPU


Central Processing Unit
ALU
ADD
XOR
ROR
Registers
0
00
1
20
2
5A
F
07
Control Unit
Program Counter
00
Instruction Register
bus
Main memory
address
cells
00
35
01
A7
02
C0
03
00
04
00
FF
00

Adding values stored in memory

Step 1. Get one of the values to be added from memory and place it in a register.

Step 2. Get the other value to be added from memory and place it in another register.

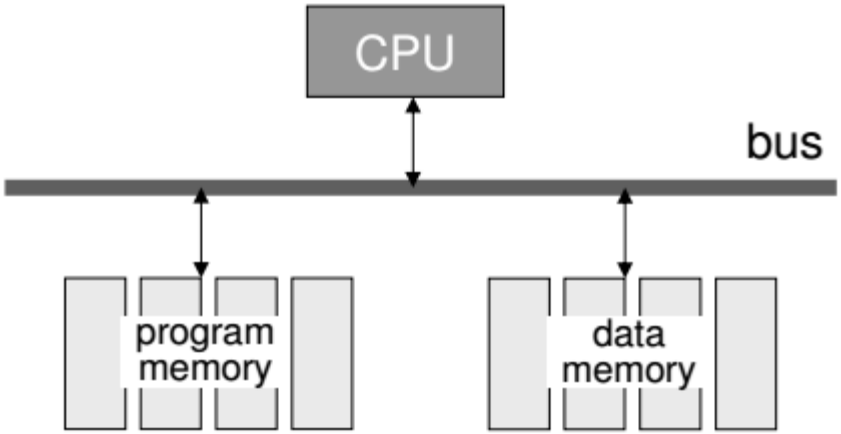
Step 3. Activate the addition circuitry with the registers used in Steps 1 and 2 as inputs and another register designated to hold the result.

Step 4. Store the result in memory.

Step 5. Stop.

Stored Program Concept

- A program can be encoded as bit patterns and stored in main memory unlike hard-wired computers in past.
- From there, the CPU can then extract the **instructions** and execute them.
- In turn, the program to be executed can be altered easily by changing data, instead of rewiring whole CPU



Terminology

- **Machine instruction:** An instruction (or command) encoded as a bit pattern recognizable by the CPU
- **Machine language:** The set of all instructions recognized by a machine

Machine Language Philosophies

- Reduced Instruction Set Computing (RISC)
  - Few, simple, efficient, and fast instructions
  - Consumes less power
  - Examples: PowerPC from Apple/IBM/Motorola and ARM
- Complex Instruction Set Computing (CISC)
  - Many, convenient, and powerful instructions
  - Consumes more power
  - Example: Intel

Machine Instruction Types

- **Data Transfer:** copy data from one location to another
- **Arithmetic/Logic:** use existing bit patterns to compute a new bit patterns
- **Control:** direct the execution of the program

Dividing values stored in memory

Step 1. LOAD a register with a value from memory.

Step 2. LOAD another register with another value from memory.

Step 3. If this second value is zero, JUMP to Step 6.

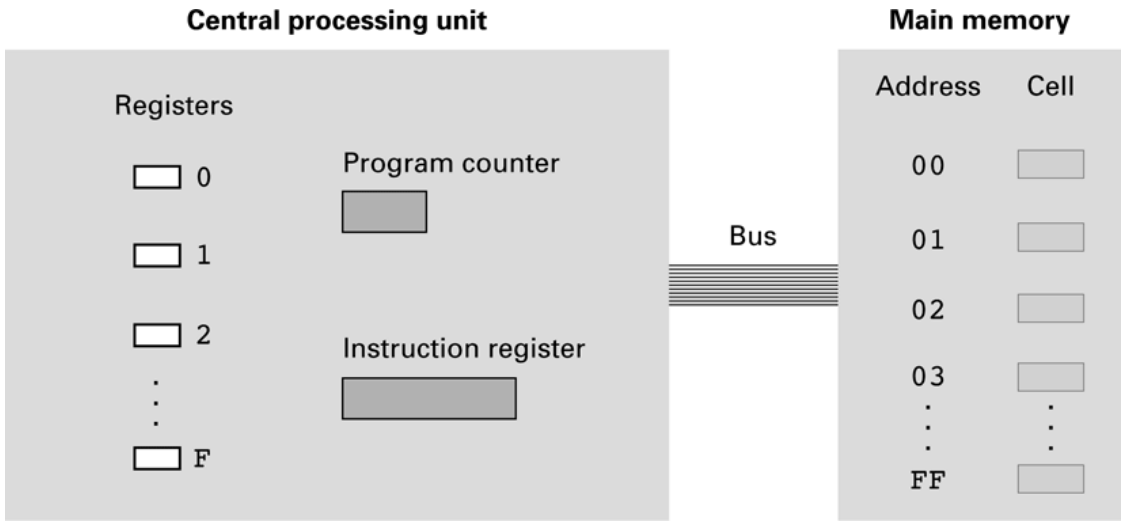
Step 4. Divide the contents of the first register by the second register and leave the result in a third register.

Step 5. STORE the contents of the third register in memory.

Step 6. STOP.

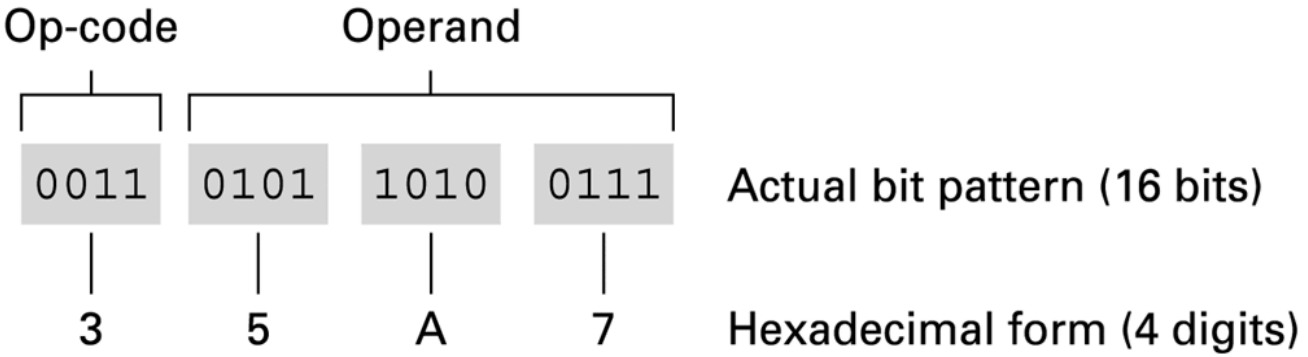
Illustrative Machine Language

- It has 16 general purpose registers (GPRs) with capacity of 8-bits and referenced using hexidecimal numbers 0 to F
- It has 256 memory cells with capacity of 8-bits and referenced using hexidecimal numbers 00 to FF



Parts of a Machine Instruction

- Each machine instruction is 16-bits longs
- **Op-code (4-bits):** Specifies which operation to execute
- **Operand (12-bits):** Gives more detailed information about the operation
  - Interpretation of operand varies depending on op-code



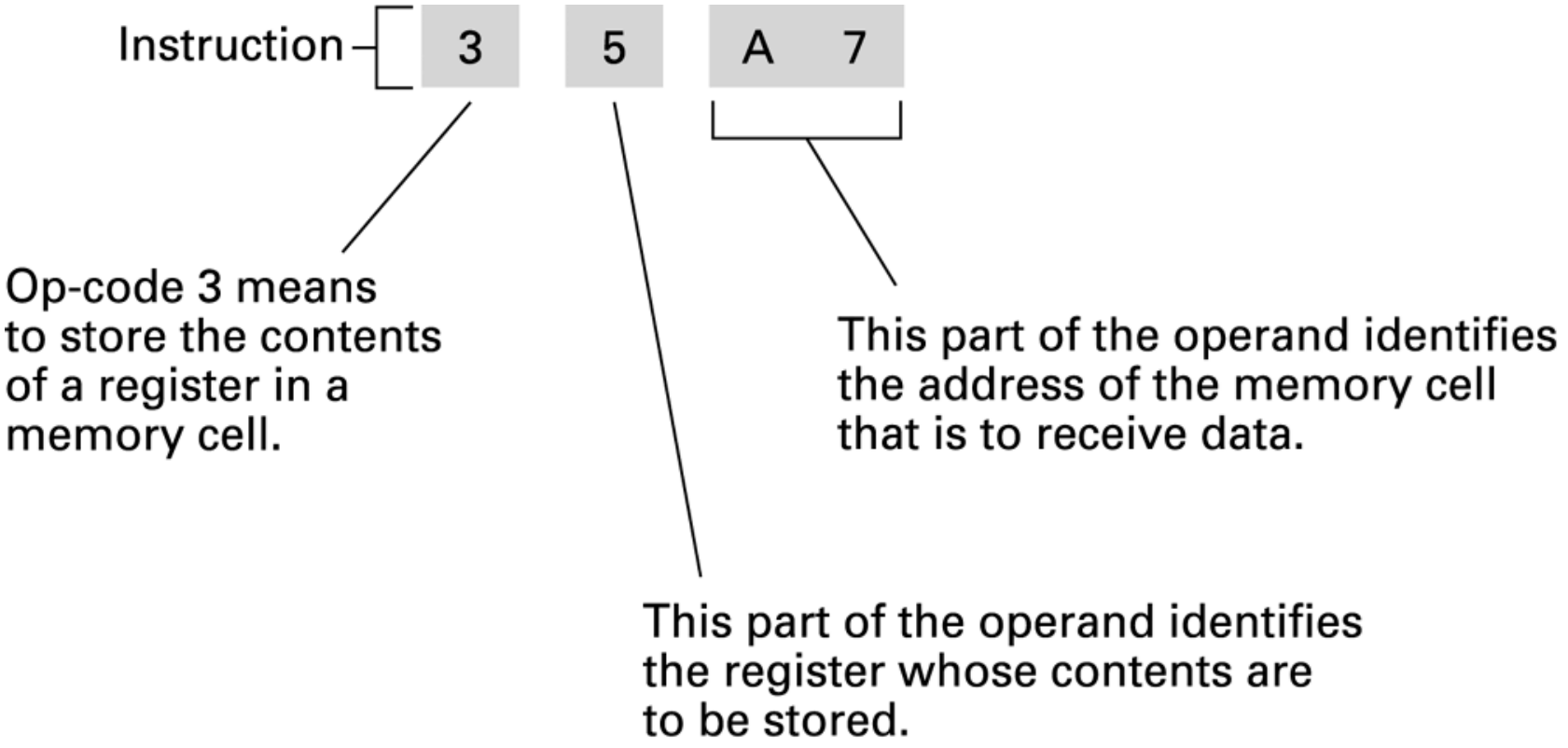
Instructions Lists

- **1: RXY:** Load the register R with the bit pattern found in the memory cell whose address is XY
- **2: RXY:** Load the register R with the bit pattern XY
- **3: RXY:** Store the bit pattern found in register R in the memory cell whose address is XY
- **4: 0RS:** Move the bit pattern found in register R to register S
- **5: RST:** Add the bit pattern found in register S and T as though they were two's complement representations and leave the result in register R
- **6: RST:** Add the bit patterns in registers S and T as though they represented values in the floating-point notation and leave the floating point result in register R
- **7: RST:** Or the bit patterns in registers S and T and place the result in register R

Instructions List (2)

- **8: RST:** And the bit patterns in register S and T and place the result in register R
- **9: RST:** Exclusive Or the bit patterns in registers S and T and place the result in register R
- **A: R0X:** Rotate the bit pattern in register R one bit to the right X times. Each time place the bit that started at the low-order end at the high order end
- **B: RXY:** Jump to the instruction located in the memory cell at address XY if the bit pattern in register R is equal to the bit pattern in register number 0. Otherwise, continue with the normal sequence of execution.
- **C:** Halt execution
- **D: RXY:** Jump to the instruction located in the memory cell at address XY if the bit pattern in register R is greater than the bit pattern in register number 0. Otherwise, continue with the normal sequence of execution. Comparisons are made in two's compliment

Decoding the instruction 35A7



Adding values stored in memory (in machine language)

Encoded instructions	Translation
156C	Load register 5 with the bit pattern found in the memory cell at address 6C.
166D	Load register 6 with the bit pattern found in the memory cell at address 6D.
5056	Add the contents of register 5 and 6 as though they were two's complement representation and leave the result in register 0.
306E	Store the contents of register 0 in the memory cell at address 6E.
C000	Halt.

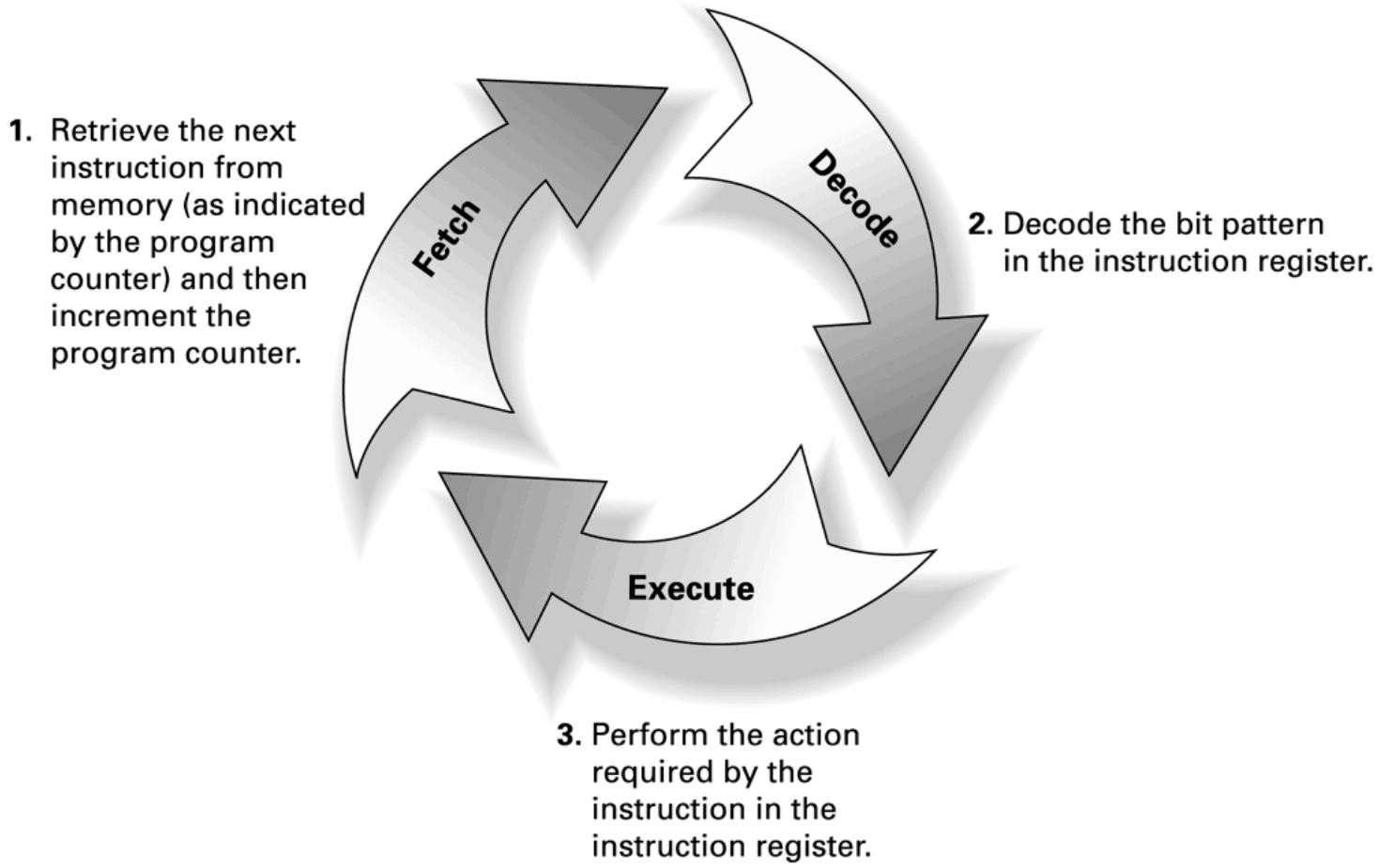
Exercise

- Convert following English instructions into Machine Language:
  - LOAD register 3 with hex value 56
  - ROTATE register 5 three bits to the right
  - AND registers A and 5, and write result to register 0
  - JUMP to memory cell E3 for further instruction if register 3 value is greater than register 0
  - JUMP to memory cell 34 for further instruction

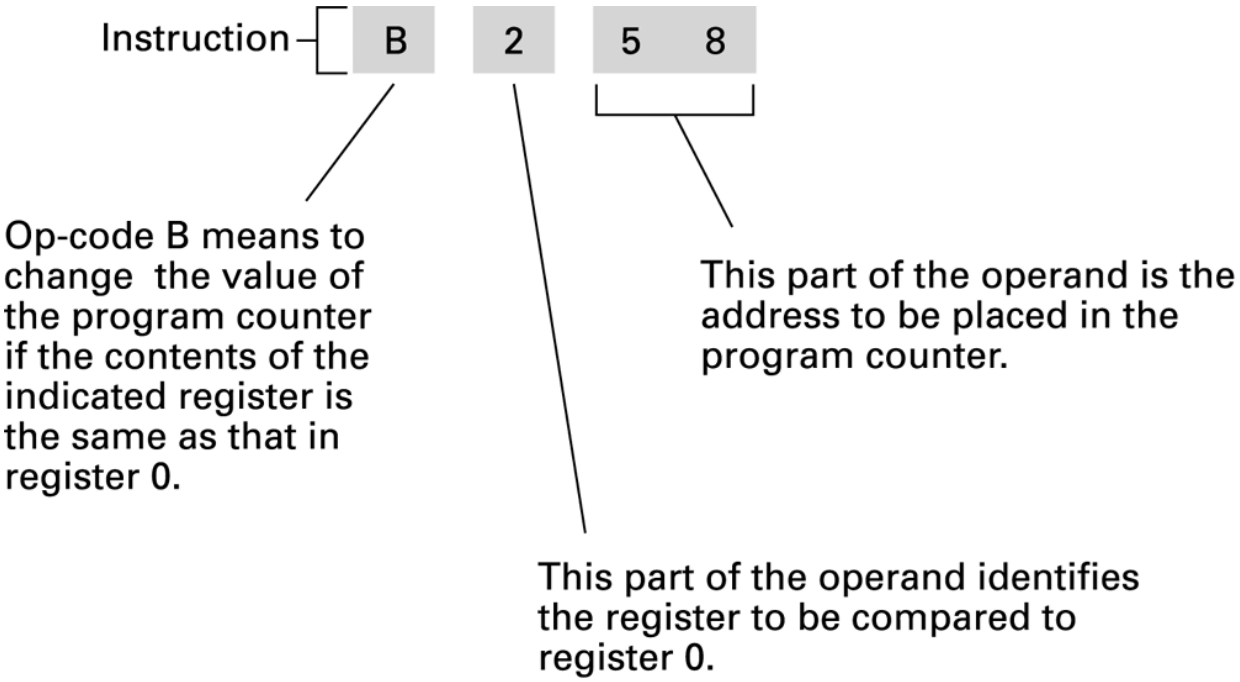
Program Execution

- Controlled by two special-purpose registers
  - Program counter: address of next instruction
  - Instruction register: current instruction
- Each machine cycle of program execution is composed of three steps:
  - **Fetch** – copies memory cells addressed by the program counter to the instruction register and increment the program counter to the next instruction in main memory
  - **Decode** – decodes the bit pattern in the instruction register to determine the operation required and the related operands
  - **Execute** – performs the operation specified by the instruction

The machine cycle

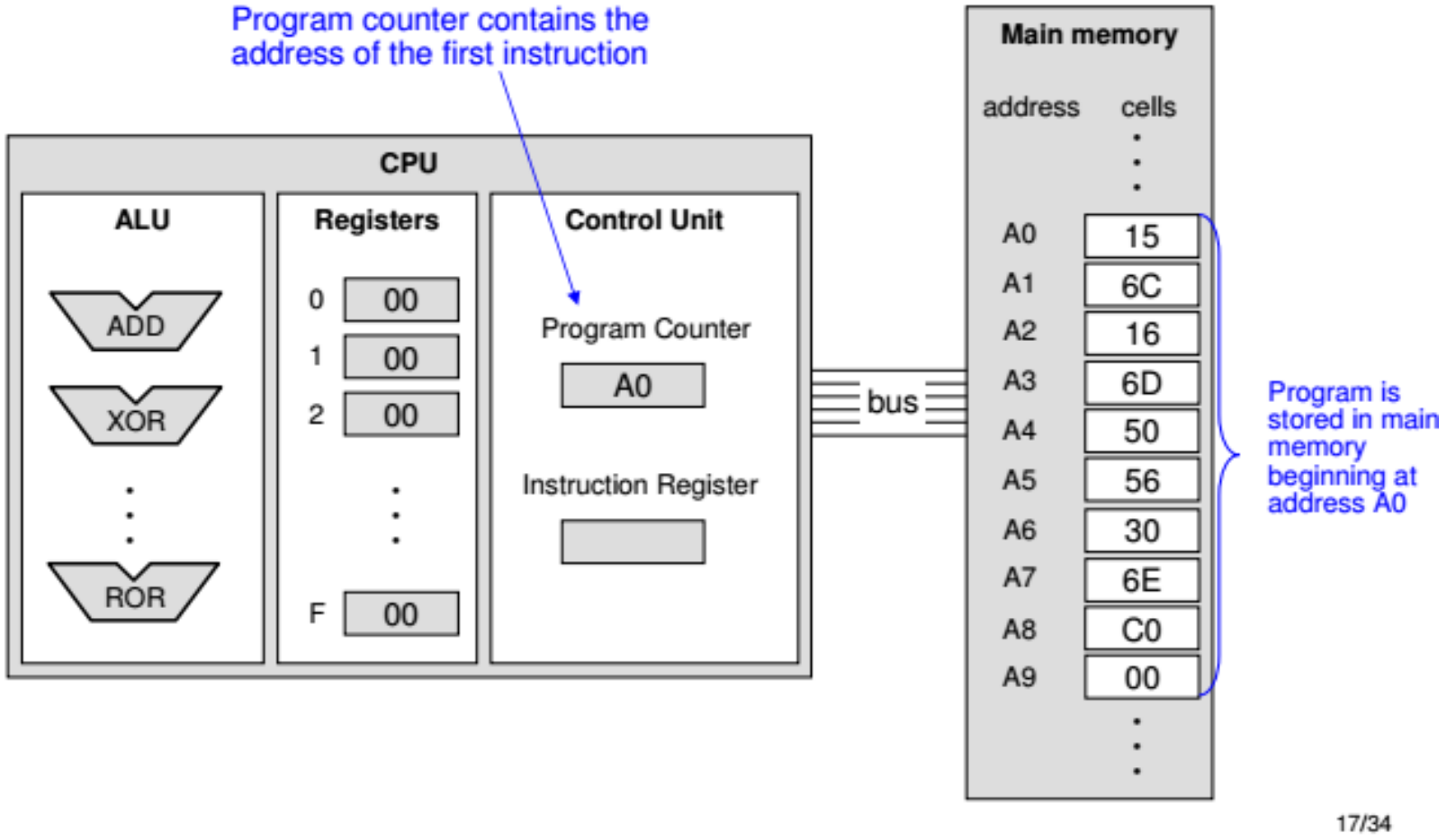


Decoding the instruction B258

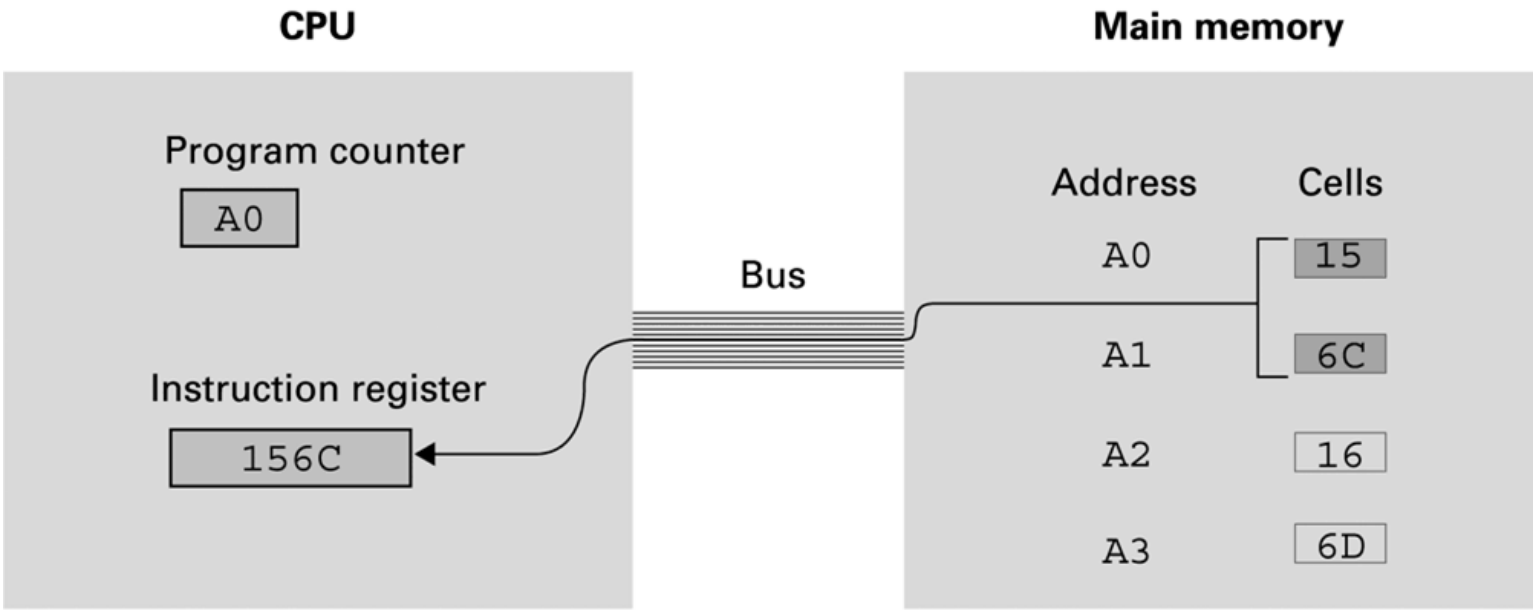


Note: while B258 simulates conditional branching, B058 simulates unconditional branching. In other words, B058 always changes the program counter to 58, because content of register 0 is equal to the content of register 0.

Adding values program stored in main memory and ready for execution

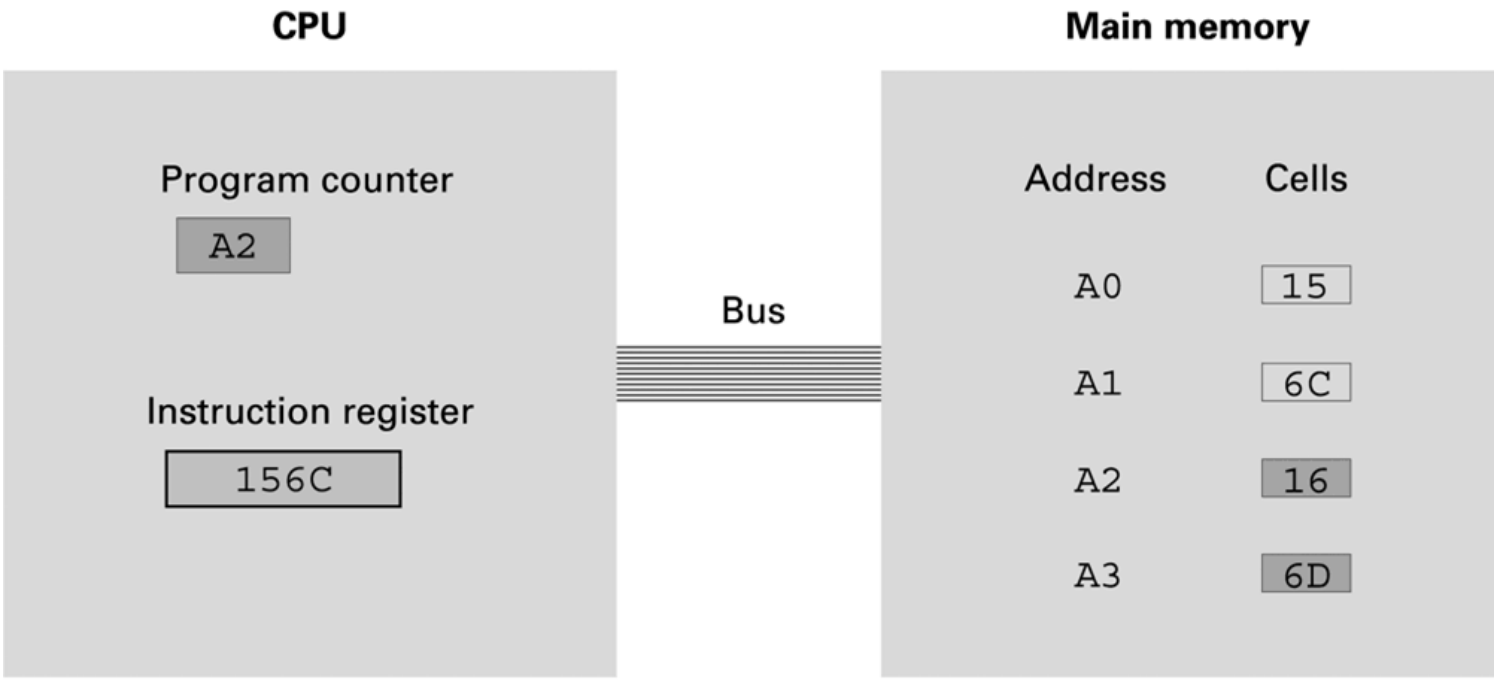


Performing the fetch step of the machine cycle



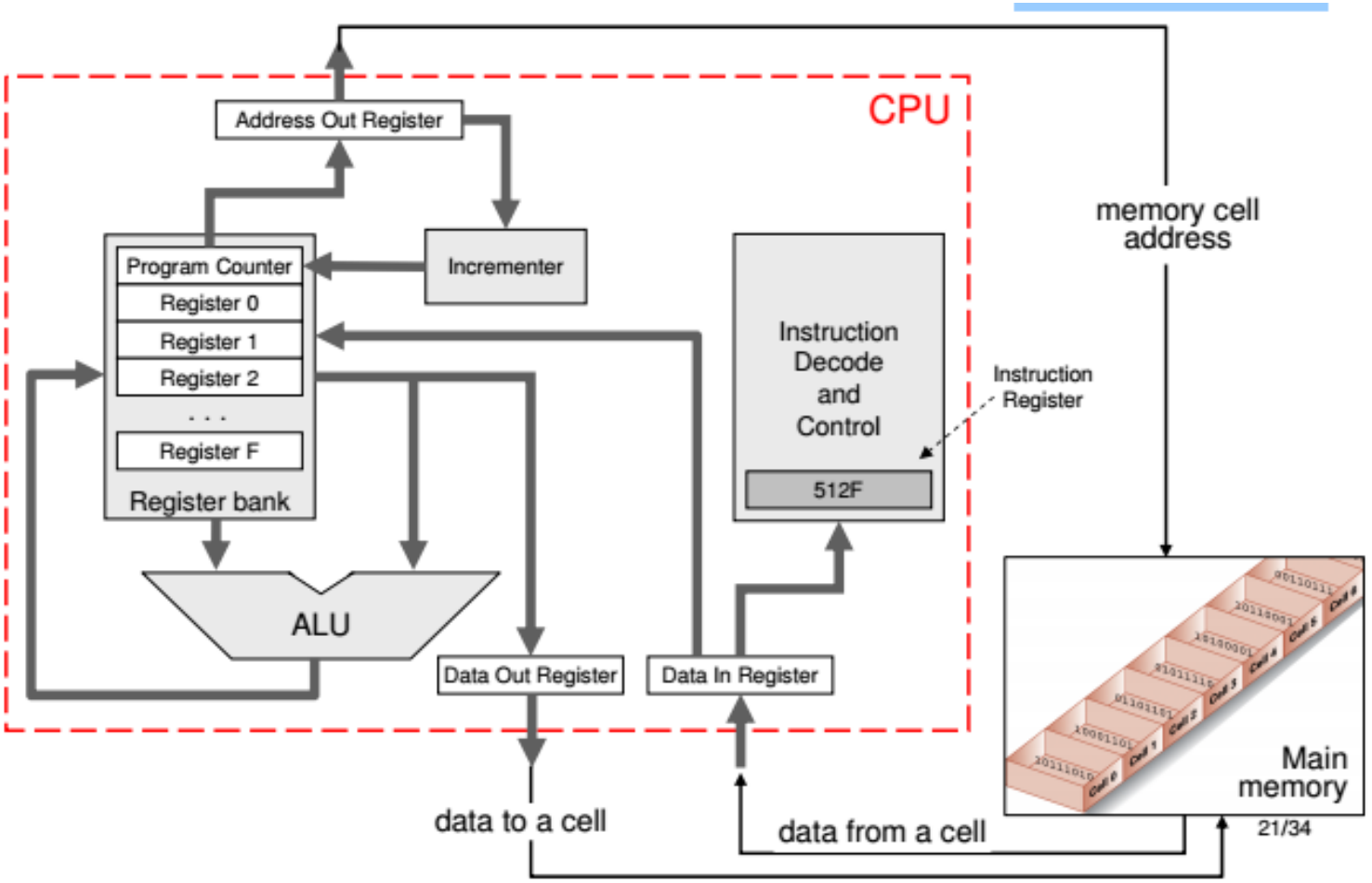
a. At the beginning of the fetch step the instruction starting at address A0 is retrieved from memory and placed in the instruction register.

Performing the fetch step of the machine cycle (continued)



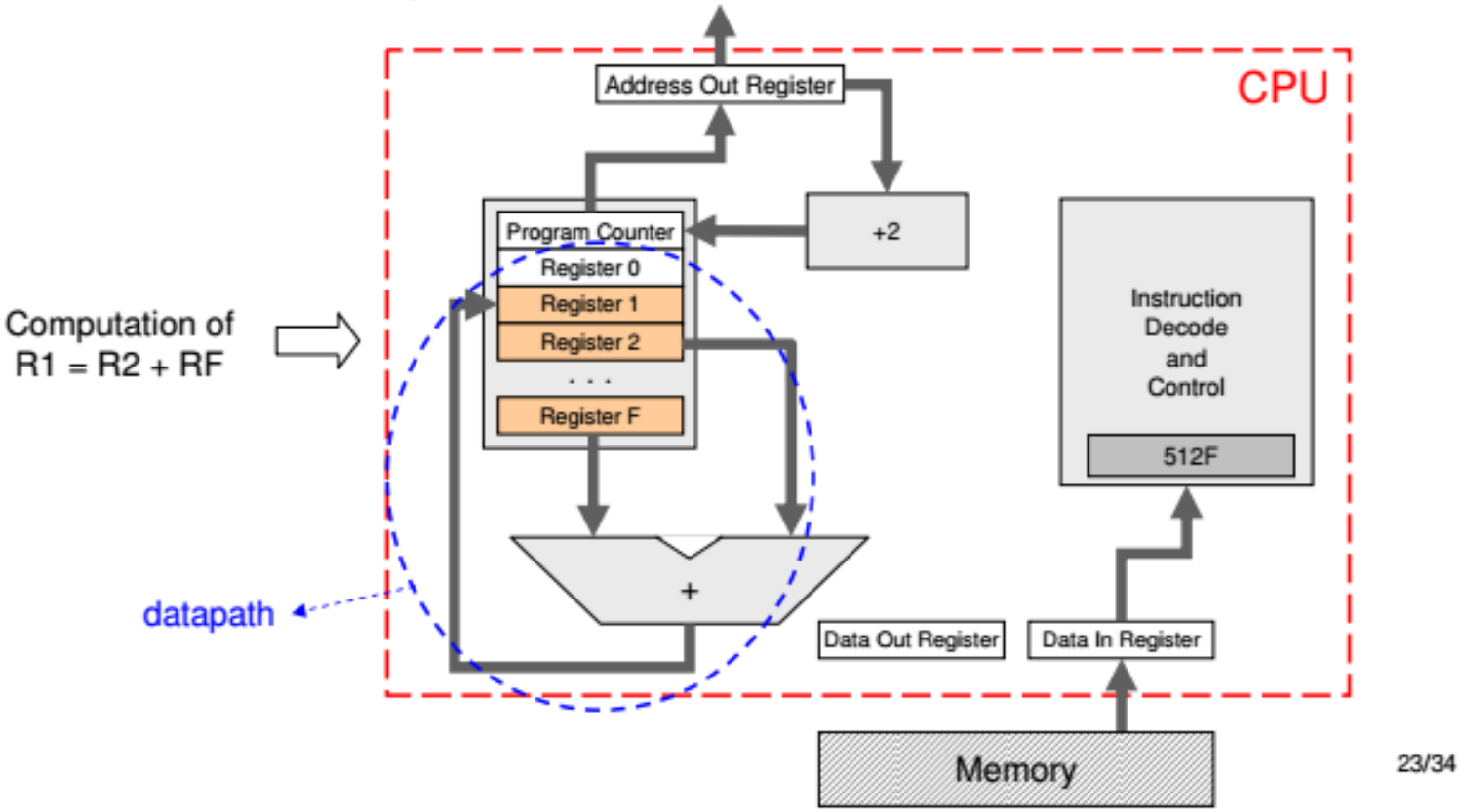
b. Then the program counter is incremented so that it points to the next instruction.

Internal Structure of a Realistic CPU



Execution of an Instruction

- After decoding of an "add" instruction, the *data path* of a CPU may become as follows:



Exercise 1

2. Suppose the memory cells at addresses B0 to B8 in the machine described in Appendix C contain the (hexadecimal) bit patterns given in the following table:

Address	Contents
B0	13
B1	B8
B2	A3
B3	02
B4	33
B5	B8
B6	C0
B7	00
B8	0F

   a. If the program counter starts at B0, what bit pattern is in register number 3 after the first instruction has been executed?
   b. What bit pattern is in memory cell B8 when the halt instruction is executed?

Exercise 2

3. Suppose the memory cells at addresses A4 to B1 in the machine described in Appendix C contain the (hexadecimal) bit patterns given in the following table:

Address	Contents
A4	20
A5	00
A6	21
A7	03
A8	22
A9	01
AA	B1
AB	B0
AC	50
AD	02
AE	B0
AF	AA
B0	C0
B1	00

When answering the following questions, assume that the machine is started with its program counter containing A4.

a. What is in register 0 the first time the instruction at address AA is executed?

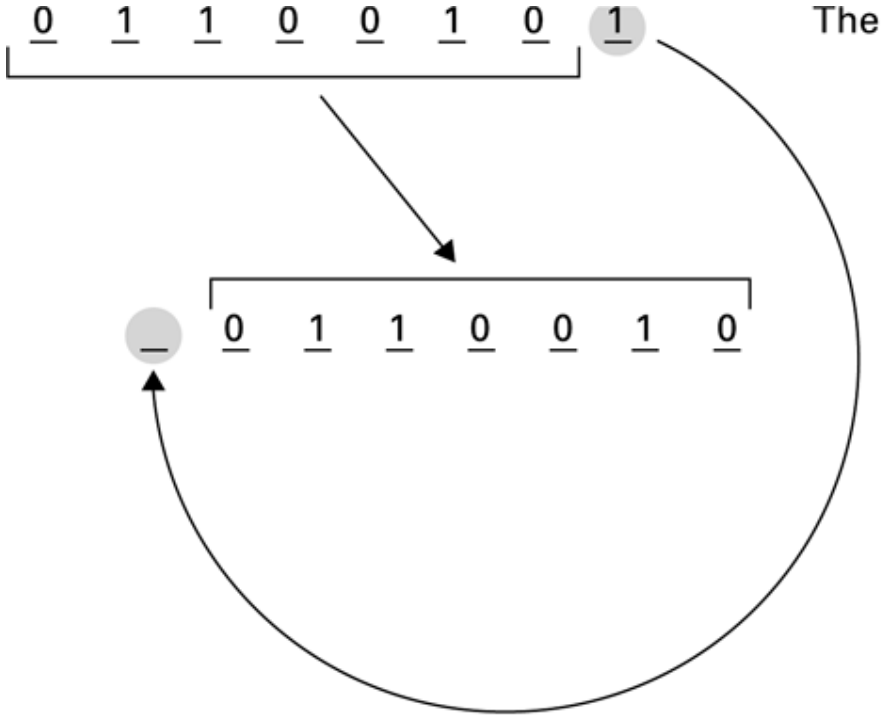
b. What is in register 0 the second time the instruction at address AA is executed?

c. How many times is the instruction at address AA executed before the machine halts?

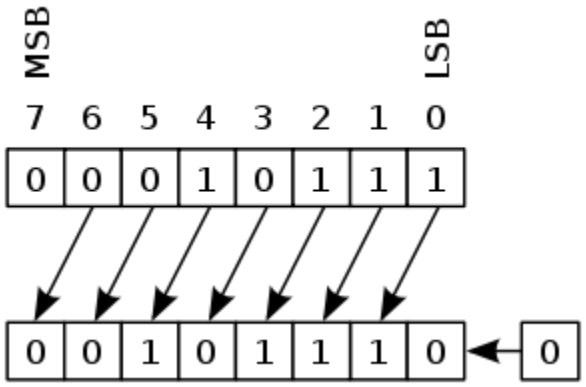
Arithmetic/Logic Operations

- Logic: AND, OR, XOR
- Masking:
  - AND can be used “to clear” required part of the bit pattern. For ex, 00001111 AND [any 8-bit] will force the leading 4-bits turn 0, and the rest get copied.
    - 00001111 AND 10101010 = 00001010
  - OR can be used “to fill” required part of the bit pattern. For ex, 00001111 OR [any 8-bit] will force the last 4-bits turn 1, and the rest get copied.
    - 00001111 OR 10101010 = 10101111
- Arithmetic: add, subtract, multiply, divide
  - Precise action depends on how the values are encoded (two’s complement versus floating-point).

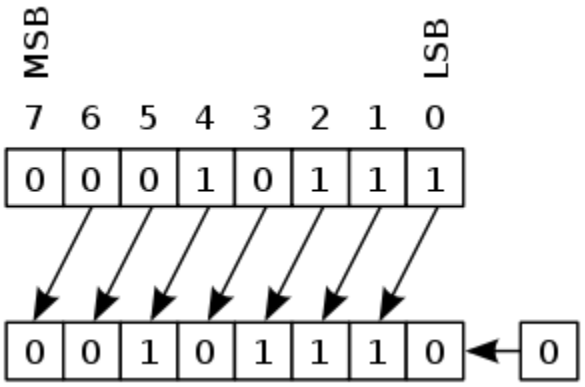
Rotating the bit pattern 65 (hexadecimal) one bit to the right



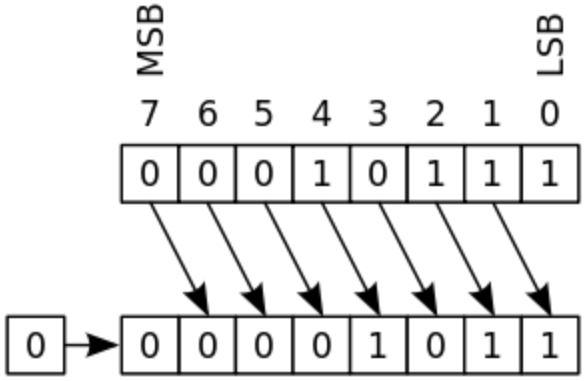
Shifting



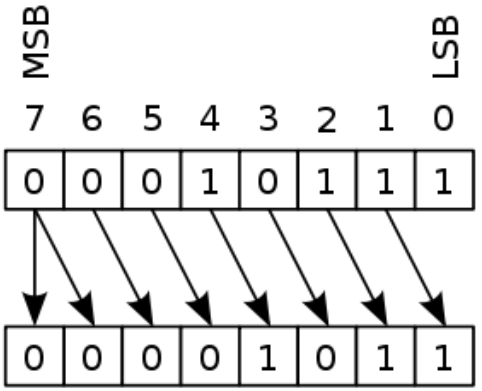
Logical left-shift



Arithmetic left-shift



Logical right-shift



Arithmetic right-shift

Bitwise Operations

- x << y: returns x with the bits shifted to the left by y places (and new bits on the right-hand-side are zeros). This is the same as multiplying x by 2^y
- x >> y: returns x with the bits shifted to the right by y places. This is the same as dividing x by 2^y
- x & y: bitwise AND
- x | y: bitwise OR
- ~ x: bitwise NOT. This is the same as -x - 1.
- x ^ y: bitwise XOR

Bit Twiddling

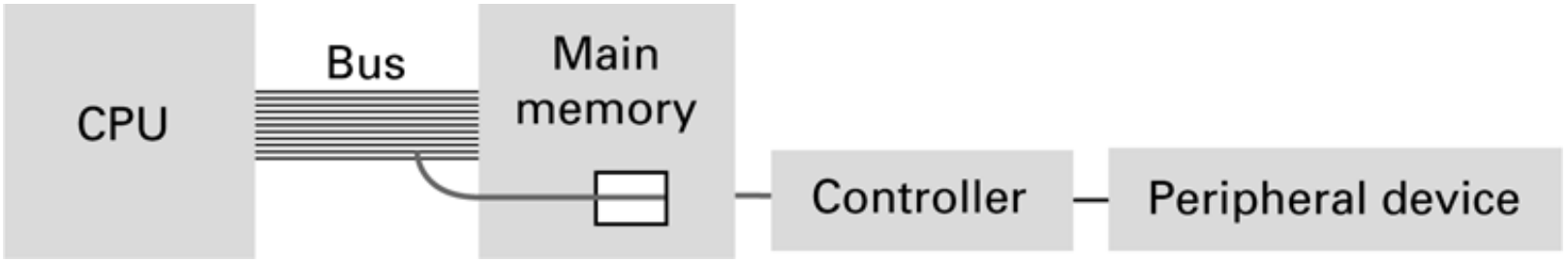
- Set n^{th} bit to 1: **x | (1<<n)**
- Set n^{th} bit to 0: **x & ~(1<<n)**
- Toggle n^{th} bit: **x ^ (1<<n)**
- Multiply by 2: **x<<1**
- Divide by 2: **x>>1**
- Increment by 1: **-~x**
- Decrement by 1: **~-x**
- If number is odd: **(x & 1) == 1**
- Two's complement negation: **x=~x+1**
- More hacks here:
  - https://github.com/keon/awesome-bits
  - https://graphics.stanford.edu/~seander/bithacks.html

Communicating with Other Devices

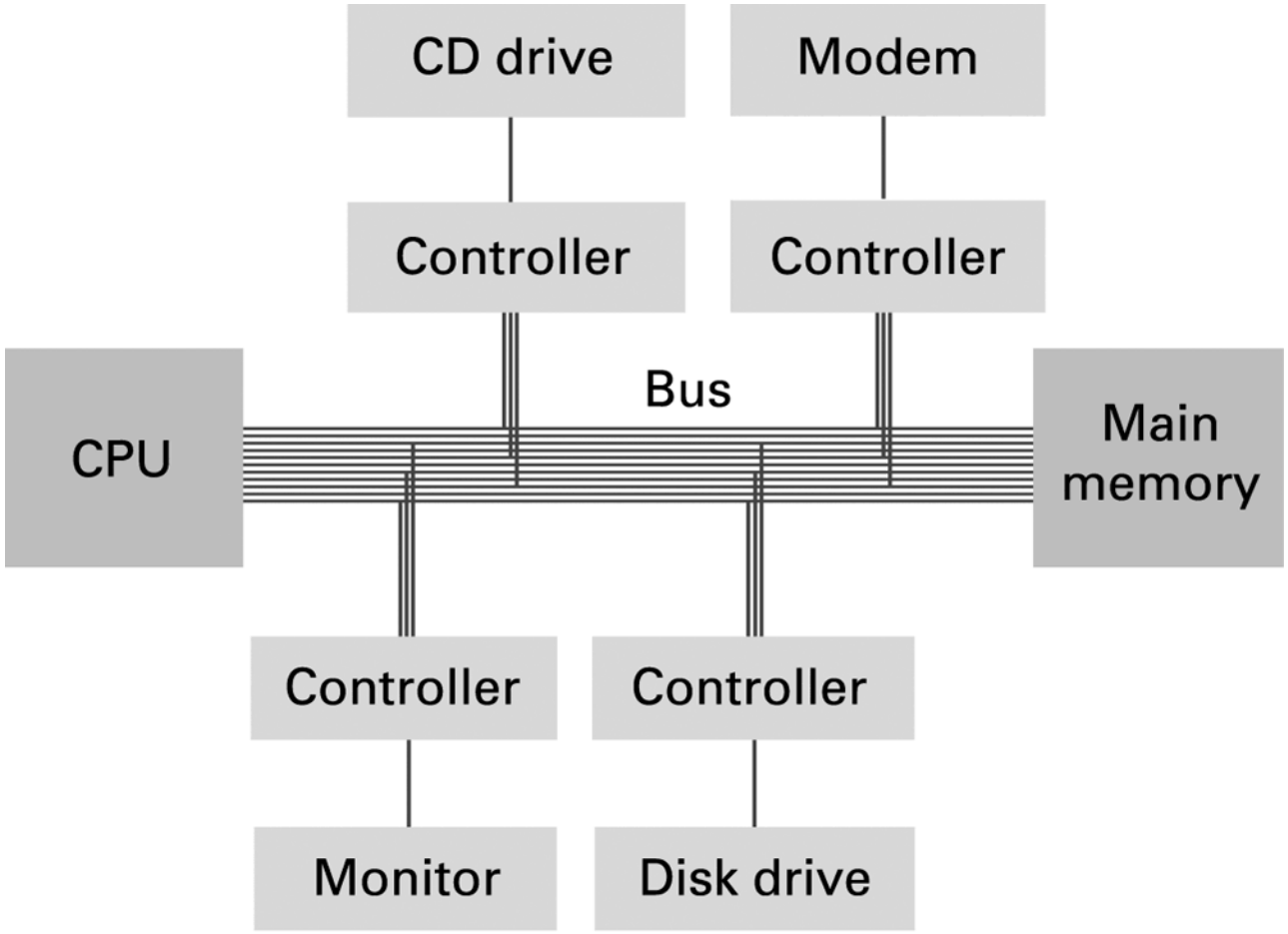
- **Controller:** An intermediary apparatus that handles communication between the computer and a device
  - Specialized controllers for each type of device
  - General purpose controllers (USB and FireWire)
- **Port:** The point at which a device connects to a computer
- **Dedicated Instruction I/O:** CPU uses dedicated I/O addresses to communicate with device controllers directly. These addresses are called I/O ports.

Memory-mapped I/O

- **Memory-mapped I/O:** CPU communicates with peripheral devices as if they were memory cells
- Memory cells identify each controller, and any information written to them gets transferred to controller, and thus to peripheral device through device driver.
- Controller's output is also written back to memory cell, so the CPU can read from it directly.



Controllers attached to a machine's bus



Communicating with Other Devices (continued)

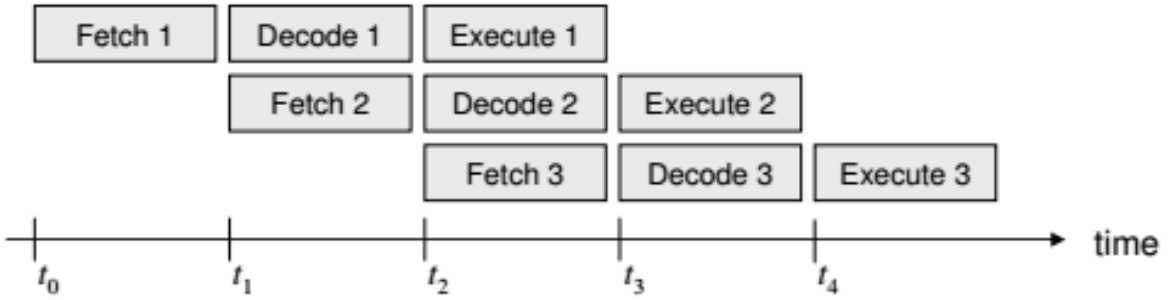
- **Direct memory access (DMA):**
  - The capability of accessing main memory directly when CPU is not using the bus.
  - Controllers have DMA
- **Von Neumann Bottleneck:**
  - Insufficient bus speed impedes performance, because both the program and the data are carried over single BUS between CPU and Memory.
- **Handshaking:** The process of coordinating the transfer of data between components

Communicating with Other Devices (continued)

- **Modem (modulation-demodulation):**
  - Converts binary data into audible tones via telephone line and vice-versa
  - In DSL, binary data is converted to non-audible tones (outside human hearing range) and carried over telephone line.
- **Parallel Communication:** Several communication paths transfer bits simultaneously.
- **Serial Communication:** Bits are transferred one after the other over a single communication path.
- **Multiplexing:** interleaving of data so that different data can be transmitted over a single communication path

Pipelining

- Pipelining: overlap steps of the CPU operation cycles



- Parallel processing: execute multiple operations simultaneously
- Parallel processing can be performed within a CPU or across CPUs

Other Architectures

- A single processing unit execute one instruction a time, which is called Single-Instruction stream Single-Data stream (SISD) architecture
- If multiple processing units (multi-CPU, multi-core, or multi-ALU) are connected to the main memory, we have parallel processing architecture:
  - Multiple-Instructions stream Multiple-Data stream (MIMD): different instructions are issued at the same time to operate on different data
  - Single-Instruction stream Multiple-Data stream (SIMD): one instruction is issued to operate on different data

Data Communication Rates

- Measurement units
  - Bps: Bits per second
  - Kbps: Kilo-bps (1,000 bps)
  - Mbps: Mega-bps (1,000,000 bps)
  - Gbps: Giga-bps (1,000,000,000 bps)
- Bandwidth: Maximum available rate

Suggested Reading/Activity

- Practice Python programming section in Chapter 2
- Install and play with Brookshear Machine Language, https://w3.cs.jmu.edu/cs101/unit03/bmachine.html
- Watch:
  - How to use Brookshear Machine, https://youtu.be/nQ5gLiKKns8
  - Impact of Computer Science on our lives, https://youtu.be/1x54GqfL3UY